Chapter 35 - Disk and File I/O

Intuition Engine exposes one disk volume through a small MMIO block. BASIC uses the same block for `LOAD`, `SAVE`, `BLOAD`, and direct-mode `DIR`. Machine code can use the registers directly, but the examples here use BASIC `POKE`, `POKE8`, `PEEK`, and `PEEK8` so they can be typed on the machine.

35.1 Names and Volume Rules

Every name in this chapter is relative to the IE disk volume. `"GAME.BAS"` names an entry at the root of the volume. `"MUSIC/TITLE.MOD"` names an entry in a volume subdirectory.

Names are rejected if they:

- begin with `/`
- contain `..`

A rejected name sets `FILE_STATUS` to `1` and `FILE_ERROR_CODE` to `3` (`FILE_ERR_PATH_TRAVERSAL`). Reads are case-insensitive when an exact-case match is not present. Writes create or replace the named entry; they do not append.

35.2 Register Block

The block starts at `$F2200` and spans `32` bytes. Registers are `32`-bit unless noted.

Address	Name	Access	Purpose
`$F2200`	`FILE_NAME_PTR`	W	Pointer to a `NUL`-terminated name string
`$F2204`	`FILE_DATA_PTR`	W	Pointer to the data buffer
`$F2208`	`FILE_DATA_LEN`	W	Byte count for write; ignored by read
`$F220C`	`FILE_CTRL`	W	Write `1` read, `2` write, `3` list
`$F2210`	`FILE_STATUS`	R	`0` OK, `1` error
`$F2214`	`FILE_RESULT_LEN`	R	Bytes transferred by read or list
`$F2218`	`FILE_ERROR_CODE`	R	Error code

`FILE_CTRL` fires the operation immediately. There is no busy bit and no interrupt. When the write to `FILE_CTRL` returns, the status registers already describe the result.

Operation codes:

Code	Name
`1`	`FILE_OP_READ`
`2`	`FILE_OP_WRITE`
`3`	`FILE_OP_LIST`

Error codes:

Code	Name	Meaning
0	FILE_ERR_OK	Success
1	FILE_ERR_NOT_FOUND	Entry does not exist
2	FILE_ERR_PERMISSION	Operation was refused
3	FILE_ERR_PATH_TRAVERSAL	Name escaped the volume

35.3 Read

Set up a read like this:

1. Put a NUL-terminated name string in memory.
2. Write that address to FILE_NAME_PTR.
3. Write the destination buffer address to FILE_DATA_PTR.
4. Write 1 to FILE_CTRL.
5. Read FILE_STATUS.

If FILE_STATUS is 0, the file bytes are in the destination buffer and FILE_RESULT_LEN is the byte count. If status is 1, read FILE_ERROR_CODE.

The reader must provide enough destination memory. The disk block does not receive a destination capacity for reads. FILE_DATA_PTR may point anywhere in active RAM. If the destination begins near the end of the low memory slice and continues into backed extended RAM, the bytes are still copied, provided every byte of the span is inside active RAM. This is a byte-copy rule. It does not make a scalar word or long PEEK/POKE valid when that one access straddles the same boundary.

FILE_DATA_LEN is write-side state. A read ignores it, even if it still contains the length from an earlier write. On a successful read, FILE_RESULT_LEN is the actual number of bytes copied. If the name is accepted but the read itself fails, FILE_RESULT_LEN is cleared to 0; use FILE_STATUS and FILE_ERROR_CODE as the final error test.

35.4 Write

Set up a write like this:

1. Put a NUL-terminated name string in memory.
2. Put the bytes to write in memory.
3. Write the name address to FILE_NAME_PTR.
4. Write the data address to FILE_DATA_PTR.
5. Write the byte count to FILE_DATA_LEN.
6. Write 2 to FILE_CTRL.
7. Read FILE_STATUS.

Writing creates the entry if it does not exist and replaces it if it does. The register block has no append mode.

35.5 List

Set up a directory listing like this:

1. Put a NUL-terminated directory name in memory. An empty string lists the root.
2. Write the name address to FILE_NAME_PTR.

3. Write a destination buffer address to `FILE_DATA_PTR`.
4. Write `3` to `FILE_CTRL`.
5. Read `FILE_STATUS`.

On success, the buffer receives sorted text with `CR` `LF` after each entry and a final `NUL` byte. Directory entries have a trailing `/`. `FILE_RESULT_LEN` counts the text bytes but not the final `NUL`.

35.6 BASIC Verbs

35.6.1 LOAD

```
LOAD "name"
```

`LOAD` reads a BASIC program from disk, tokenises it, makes it the current program, and clears variables. If the entry is not found, BASIC prints `?FILE NOT FOUND` and keeps the previous program.

35.6.2 SAVE

```
SAVE "name"
```

`SAVE` writes the current BASIC program as detokenised numbered text. The saved text round-trips through `LOAD`.

35.6.3 BLOAD

```
BLOAD "name", addr
```

`BLOAD` reads raw bytes into memory at `addr`. It does not tokenise and it does not clear variables.

35.6.4 DIR

```
DIR
DIR "subdir"
```

`DIR` is a direct-mode command. It lists the root or the named directory and prints entries separated by `CR` `LF`. Its output depends on the current disk volume, so it is shown here as a syntax template rather than a transcript with fixed expected output.

35.7 Typed MMIO Example

This BASIC listing writes two bytes to `NOTE.TXT`, clears the buffer, reads the file back, and prints the status and byte values.

```
10 REM NAME BUFFER AND DATA BUFFER
20 N=&H00720000:D=&H00720100
30 REM "NOTE.TXT",0
40 POKE8 N,78:POKE8 N+1,79:POKE8 N+2,84:POKE8 N+3,69
50 POKE8 N+4,46:POKE8 N+5,84:POKE8 N+6,88:POKE8 N+7,84
60 POKE8 N+8,0
70 REM FILE DATA "IE"
80 POKE8 D,73:POKE8 D+1,69
90 POKE &H000F2200,N
100 POKE &H000F2204,D
110 POKE &H000F2208,2
120 POKE &H000F220C,2
130 PRINT "WRITE ";PEEK(&H000F2210)
140 REM CLEAR THE BUFFER AND READ THE FILE BACK
150 POKE8 D,0:POKE8 D+1,0
160 POKE &H000F220C,1
170 PRINT "READ ";PEEK(&H000F2210)
180 PRINT "LEN ";PEEK(&H000F2214)
190 PRINT PEEK8(D);PEEK8(D+1)
```

Expected result:

```
WRITE 0
READ 0
LEN 2
73 69
```

Lines `40` to `60` build the byte string `"NOTE.TXT",0`. Lines `80` to `120` provide the two data bytes and fire the write operation. Line `150` clears the RAM buffer so the readback cannot be mistaken for leftover data. Lines `160` to `190` fire the read, print the byte count, and then print the two returned bytes.

35.8 Small-CPU Access

Full-address CPUs write the register block directly. The 6502 and Z80 use their documented MMIO translation apertures to reach the same block. The block also accepts byte-width writes: four writes to consecutive byte offsets compose a `32`-bit register value in little-endian order.

Writing byte `0` of `FILE_CTRL` with `1`, `2`, or `3` triggers the matching operation. Writes to the upper bytes of `FILE_CTRL` do not trigger an operation.

35.9 Limits and Side Effects

Names longer than `255` bytes are truncated before lookup. Reads ignore `FILE_DATA_LEN`; writes use it as the number of bytes to copy from `FILE_DATA_PTR`. Successful reads and lists set `FILE_RESULT_LEN` to the number of bytes returned. Lists set `FILE_RESULT_LEN` to `0` on failure. Reads whose path is accepted but whose file cannot be read also set `FILE_RESULT_LEN` to `0`. After a failed write, do not rely on `FILE_RESULT_LEN`.

Read destinations are ordinary bus addresses. They may be in low RAM or backed extended RAM, and the transfer may cross that boundary because the file block writes one byte at a time. The program must still choose an address range that is large enough for the file.

The block is synchronous and single-operation. Program code should not change `FILE_NAME_PTR`, `FILE_DATA_PTR`, or `FILE_DATA_LEN` while an operation is in progress, although in normal use there is no observable busy interval.